

Over-Squashing vs. Adversarial Robustness in GNNs: An “Add vs. Remove Edge” Investigation

Supanut Kompayak (st126055) Dechathon Niamsa-Ard (st126235)
Deep Learning Course Project, Asian Institute of Technology
{st126055, st126235}@ait.asia

Final Report — July 2, 2026

Abstract

Graph neural networks (GNNs) suffer from two seemingly unrelated structural problems: *over-squashing*, where long-range information is compressed through bottlenecks and lost, and *adversarial fragility*, where a few maliciously edited edges flip predictions. Both concern how sensitive a node’s output is to structural change elsewhere. As a learning project, we *investigate* whether methods that mitigate over-squashing also help adversarial robustness, through the lens of **adding vs. removing edges**: over-squashing rewiring methods either *add* edges (FoSR) or *remove* them (spectral pruning), while adversarial attacks add edges and adversarial defenses (GCN-Jaccard) remove them. On Cora and Citeseer node classification we compare a vanilla GCN, an add-based rewiring (FoSR), a remove-based spectral pruning (Jamadandi ProxyDelete), and two purpose-built defenses (GCN-Jaccard, GCN-SVD) under the Metattack poisoning attack, reporting clean accuracy, robust accuracy, and the robustness gap, plus a Jacobian-based over-squashing probe, an edge-overlap mechanism analysis, and a defense-aware attack. We report only real, reproducible numbers; negative results are acceptable. We find that over-squashing mitigations do *not* transfer to robustness — both add-based FoSR and spectral remove-based ProxyDelete behave like an undefended GCN, while only the attack-aware defenses (GCN-Jaccard, GCN-SVD) improve robustness — indicating that the editing *criterion*, not the add/remove direction, governs robustness on this testbed.

1 Introduction

Message-passing GNNs propagate information hop-by-hop along edges. Two known failure modes both stem from how information flows through graph structure. **Over-squashing** [1, 12, 3]: reaching distant nodes requires many layers, but the exponentially growing long-range signal is squeezed through structural bottlenecks into a fixed-size vector and lost — the output is *not sensitive enough* to distant signal. **Adversarial fragility** [14, 15]: an attacker adds or removes a few edges and message passing pulls in the bad signal, flipping the prediction — the output is *too sensitive* to malicious structural change. Both are about the sensitivity of a node’s output to structural change elsewhere; this conceptual bridge is what we investigate, not a claim we prove.

The sharper framing we use is **add vs. remove edge**. Over-squashing rewiring mostly *adds* edges to raise the spectral gap and relieve bottlenecks (FoSR [8], SDRF [12]); adversarial attacks typically *add* edges between dissimilar nodes; adversarial defenses *remove* suspicious edges (GCN-Jaccard [13], Pro-GNN [7]). Thus adding edges to fix over-squashing might *hurt* robustness (more edges to exploit), while a *remove*-based over-squashing method — spectral pruning [6] — might transfer to robustness because it removes edges like a defense does. “Add vs. remove: which helps robustness, and why?” is the spine of this project.

2 Related Work and Positioning

The building blocks are established. Over-squashing is characterized by Alon and Yahav [1], Topping et al. [12] and Di Giovanni et al. [3], and mitigated by spectral rewiring that *adds* edges (FoSR [8]) or *deletes* them [6]. Structural adversarial attacks and defenses are equally well studied: Metattack [14] and Nettek [15] perturb the graph, and GCN-Jaccard [13] defends by pruning edges between dissimilar nodes — Wu et al. [13] report that attacks “tend to connect the target node to nodes with different features” and that the defense removes “edges that connect nodes with low similarity.”

What is missing is the *link* between the two lines of work, which motivates our study. To our knowledge the interaction is unstudied: a current, comprehensive rewiring survey [2] does not mention adversarial robustness at all, and the spectral-rewiring methods we use [8, 6] neither evaluate nor discuss it; conversely the GNN-robustness literature [13, 4] does not engage over-squashing mitigation. The question is natural because the operations *collide*: over-squashing rewiring adds edges to raise connectivity — structurally the same move an attacker makes — so whether it helps or hurts robustness is a priori unclear. A recent position paper [10] independently argues over-squashing mitigations bring limited practical gains, echoing our robustness-side result, and the closest related method, GraphTorque [5], improves robustness by edge *removal* — consistent with our finding that removal, not addition, helps. We contribute the controlled add-vs-remove study that connects these threads; as a learning project, the contribution is the question and its controlled empirical answer, not any new method.

3 Research Questions

- **RQ2 (lead).** Do over-squashing mitigations (add-based FoSR vs. remove-based spectral pruning) improve adversarial robustness relative to (a) an undefended GCN and (b) a purpose-built defense (GCN-Jaccard), and at what cost to clean accuracy?
- **RQ1 (secondary).** Does structural perturbation measurably worsen an over-squashing measure? Is there a relationship between perturbation strength and long-range information loss?
- **RQ3 (stretch).** Can a single measure (Jacobian sensitivity) capture both over-squashing severity and adversarial fragility on the same scale?

4 Data

Cora is the workhorse; everything else is optional. All statistics are confirmed at load time.

5 Methods and Approach

Backbone. GCN [9] (2 layers, hidden 16, dropout 0.5). GAT/GraphSAGE are stretch.

Dataset	Task	Size	Role
Cora (Planetoid)	node clf.	2,708 nodes / 5,278 undir. edges / 7 cls / 1,433 feat	primary
Citeseer (Planetoid)	node clf.	3,327 / 4,552 / 6 / 3,703	2nd setting
Peptides-func (LRGB)	graph clf. (AP)	15,535 graphs, ~ 150 avg nodes	stretch
ZINC (subset)	graph reg. (MAE)	12,000 graphs, ~ 23 avg nodes	stretch

Table 1: Datasets. Cora is the primary node-level and standard adversarial-attack testbed.

The method table (the design). Each method occupies a position on the add/remove axis:

Method	Targets	Edge op	Role
GCN (vanilla)	—	none	baseline
FoSR	over-squashing	add	the “add” arm
Spectral pruning (ProxyDelete)	over-squashing	remove	the “remove” arm
GCN-Jaccard	adversarial defense	remove	defense ref. (similarity)
GCN-SVD	adversarial defense	low-rank	defense ref. (low-rank)
DropEdge	regularization	remove (random)	random-removal sanity baseline

Table 2: *FoSR* vs. *ProxyDelete* isolates the add/remove *direction*; *ProxyDelete* vs. the defenses isolates the removal/edit *criterion* (spectral vs. similarity vs. low-rank); *all* vs. *vanilla* asks whether any of this helps robustness, and at what clean-accuracy cost. GCN-SVD is a second, mechanism-distinct defense reference.

Attack and defense. Metattack [14], a global meta-learning poisoning attack, at perturbation rates $\{0.05, 0.10\}$. Defense reference: GCN-Jaccard [13]. Both via DeepRobust [11]. Remove-based over-squashing arm: ProxyDelete spectral pruning [6] (deletes edges to maximize the spectral gap). Add-based arm: FoSR [8], adapted from graph- to node-classification.

Pipeline order (the operational spine). Metattack is a *poisoning* attack, so operation order is part of the design:

$$\text{clean graph} \xrightarrow{\text{Metattack}} \text{poisoned } G' \xrightarrow{M} G'' \xrightarrow{\text{train GCN}} \text{eval.}$$

The transform $M \in \{\text{none, FoSR, ProxyDelete, GCN-Jaccard}\}$. A method changes the graph even with no attack, giving four conditions per method: clean; clean+ M ; poisoned; poisoned+ M .

Over-squashing probe. Jacobian sensitivity $\partial \mathbf{h}_v / \partial \mathbf{x}_u$ for distant pairs (u, v) [3], via autograd (scoped to a few far pairs first).

6 Experimental Setup

All robustness experiments use DeepRobust’s Cora with the standard `netattack` split (largest connected component, 2485 nodes). The poisoned adjacency is generated once per perturbation rate (Metattack, Meta-Self, attack seed 15) and cached; every method reuses it. Reported accuracies are mean $\pm$ std over 3 training seeds on the fixed poisoned graph. Backbone hyperparameters: hidden 16, dropout 0.5, lr 0.01, weight decay 5×10^{-4} , 200 epochs. Rewiring/removal methods share a matched edge budget for the add/remove comparison; GCN-SVD uses rank $k=50$. All computation runs on CPU. The main attacker is non-adaptive (it attacks a vanilla GCN surrogate, unaware of the defense); we probe this assumption directly with a heuristic defense-aware attack (§7, “adaptive attacker”).

7 Results

Baseline. On the standard Planetoid public split the vanilla GCN reaches $80.30 \pm 0.47\%$ (5 seeds). On the DeepRobust `netattack` split (2485 nodes, 3 seeds) it reaches $83.50 \pm 0.46\%$ clean, falling to 77.05% and 71.85% under Metattack at $\text{ptb} = 0.05$ and 0.10 (Table 3).

Headline: add vs. remove under attack. Table 3 reports the full grid, and the result is decisive — and largely negative for the over-squashing hypothesis. *Only the two purpose-built defenses improve robustness.* The undefended GCN, random removal (DropEdge), add-based rewiring (FoSR) and spectral removal (ProxyDelete) all cluster together (robust accuracy 76.7–77.1% at ptb 0.05 and 71.0–71.9% at ptb 0.10), while GCN-Jaccard and GCN-SVD stand clearly apart. Jaccard has the best low-perturbation robust accuracy (78.84%) at a small clean cost (~ 1.3 pts); GCN-SVD, via low-rank filtering, is the most attack-immune (gap 0.4/2.3; the best robust accuracy at ptb 0.10, 75.29%) though at a larger clean cost (~ 6 pts). FoSR and ProxyDelete show marginally smaller *gaps* than the vanilla GCN only because their clean accuracy is lower — their robust accuracy is not improved — so the gap must be read alongside robust accuracy. Crucially, *the add/remove direction does not determine robustness*: both over-squashing methods fail to defend whether they add (FoSR) or remove (ProxyDelete) edges. What separates the defenses is that both are *attack-aware* — Jaccard removes the dissimilar-feature edges the attacker inserts, SVD filters the high-rank perturbation — whereas the over-squashing methods optimize a structural quantity (the spectral gap) unrelated to the attack.

Method	Clean	Robust@0.05	Robust@0.10	Gap@0.05	Gap@0.10
GCN (vanilla)	$83.50 \pm .46$	$77.05 \pm .51$	$71.85 \pm .72$	6.46	11.65
FoSR (add)	$82.53 \pm .13$	$76.78 \pm .60$	$70.96 \pm .55$	5.75	11.57
ProxyDelete (spectral rm.)	$82.60 \pm .07$	$76.71 \pm .52$	$71.68 \pm .68$	5.89	10.92
DropEdge (random rm.)	$83.17 \pm .61$	$76.98 \pm .40$	$71.40 \pm .87$	6.19	11.77
GCN-Jaccard (similarity rm.)	$82.16 \pm .39$	$78.84 \pm .27$	$74.45 \pm .68$	3.32	7.71
GCN-SVD (low-rank)	$77.58 \pm .41$	$77.20 \pm .14$	$75.29 \pm .60$	0.39	2.30

Table 3: Headline grid on Cora (test acc %, `netattack` split, Metattack, 3 seeds). Lower gap = more robust. Top block: undefended GCN and the over-squashing/random arms; bottom block: the two purpose-built defenses. Only the defenses separate from the vanilla-GCN level.

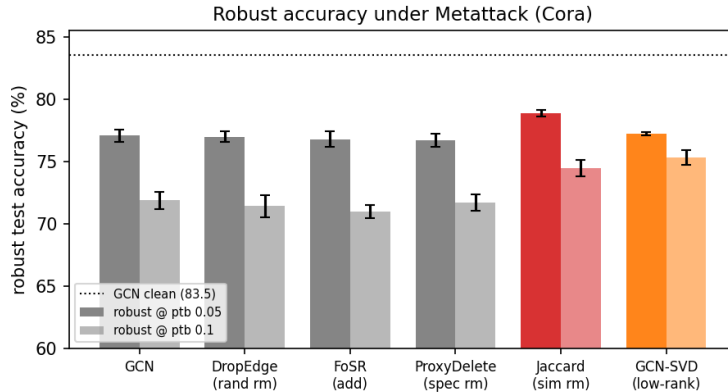


Figure 1: Robust accuracy per method under Metattack (Cora). The over-squashing and random arms cluster at the vanilla-GCN level; only GCN-Jaccard (red) separates. Dotted line: clean GCN accuracy.

Why does Jaccard win? (mechanism). Metattack here is almost purely edge-*addition* (it inserts 253/503 edges at ptb 0.05/0.10 and deletes 0/3), which validates the add/remove framing. Table 4 then measures how many of those inserted edges each method deletes. The story is mechanistic: GCN-Jaccard removes about *half* of the attacker’s edges (recall 0.50, $\sim 3.6\times$ the chance rate of a random remover) — it literally undoes the attack. Spectral pruning removes attack edges only slightly above chance (0.19 vs. 0.12), random DropEdge sits exactly at chance, and FoSR (which only adds) removes none. Robustness therefore tracks *how many attacker edges a method deletes*, and only the similarity criterion deletes them at scale.

Method (ptb 0.10)	# removed	attack edges rm.	recall	chance
GCN-Jaccard	797	250	0.50	0.14
ProxyDelete (spectral)	643	93	0.19	0.12
DropEdge (random)	643	59	0.12	0.12
FoSR (add)	0	0	0.00	—

Table 4: Edge-overlap mechanism (Cora, ptb 0.10). *Recall* = fraction of the attacker’s inserted edges the method deletes; *chance* = recall of a random remover of the same size. Only Jaccard removes attack edges far above chance — explaining why only it defends.

Do the methods raise the spectral gap? (sanity). The over-squashing methods do exactly what they claim — it just does not help. On clean Cora the normalized-Laplacian spectral gap (λ_2) is 0.005; the poisoning attack itself *raises* it to 0.016 (adding edges improves connectivity), and FoSR raises it further to 0.054, a $3.3\times$ increase over the poisoned graph. Yet neither buys robustness — the attack lowers accuracy and FoSR leaves it unchanged. Raising the spectral gap, the explicit objective of over-squashing rewiring, is thus uncorrelated (here even anti-correlated) with robustness, so the methods are working as intended on the wrong quantity. (Removal-based methods can fragment the graph, making λ_2 degenerate, so we make this point on the add side; the removal side is covered by the edge-overlap analysis above.)

Over-squashing probe (RQ1). Figure 2 plots the mean Jacobian sensitivity $\|\partial h_v / \partial x_u\|_F$ of a 6-layer GCN against graph distance $d(u, v)$. The over-squashing signature is unmistakable: sensitivity decays roughly geometrically, about $5\times$ per hop (from 4.0 at $d=1$ to 7×10^{-4} at $d=6$ on the clean graph), so distant nodes have almost no influence on a node’s output. Comparing the clean and poisoned (ptb = 0.10) graphs, the per-hop decay rate is essentially unchanged; poisoning lowers the overall magnitude but does not steepen the long-range decay. Thus structural attack does not worsen the over-squashing *measure* itself at these perturbation levels — consistent with the RQ2 finding that the two phenomena are governed by different factors. (Absolute magnitudes depend on the separately trained weights, so the decay *slope* is the reliable comparison; a weight-controlled variant is a natural follow-up.)

Second dataset (Citeseer). The pattern generalizes (Table 5). Citeseer is naturally more robust here — the attack opens only a 1.9/4.2-point gap on the vanilla GCN — but the ordering is identical: GCN-Jaccard is again strongest, with the highest robust accuracy (70.97/68.40%) and smallest gap (1.03/3.59), while spectral removal (ProxyDelete, 68.82/66.82) and add-based rewiring (FoSR, 69.33/65.36) do not help — FoSR again hurts at the higher perturbation rate. Random removal (DropEdge) tracks the vanilla GCN. As on Cora, only similarity-targeted removal improves robustness.

An adaptive (defense-aware) attacker. Our attacker so far is non-adaptive. Since Jaccard removes edges between *dissimilar* nodes, a defense-aware attacker should instead add edges between feature-*similar* but differently-labelled nodes, which the filter cannot remove. We built exactly such

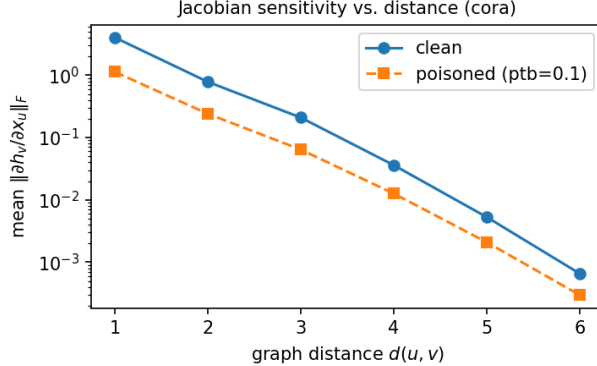


Figure 2: Jacobian sensitivity $\|\partial h_v / \partial x_u\|_F$ vs. graph distance (6-layer GCN, Cora; log scale). The $\sim$ geometric decay is the over-squashing signature; clean and poisoned graphs share the same decay rate.

Method	Clean	Robust@0.05	Robust@0.10	Gap@0.05	Gap@0.10
GCN (vanilla)	71.84	69.94	67.69	1.90	4.15
FoSR (add)	71.23	69.33	65.36	1.90	5.86
ProxyDelete (remove)	71.60	68.82	66.82	2.78	4.78
GCN-Jaccard (remove)	72.00	70.97	68.40	1.03	3.59
DropEdge (random)	71.64	69.67	67.65	1.97	3.99

Table 5: Citeseer headline grid (test acc %, net attack split, Metattack, 3 seeds). The Cora conclusion holds: only GCN-Jaccard improves robustness.

a heuristic evasion attack. It succeeds at the filter level — Jaccard removes 0% of its inserted edges — so the defense is, in principle, evadable. But the evasion is largely self-defeating: forcing edges to be similarity-high drastically weakens the attack, dropping vanilla-GCN accuracy by only 0.7/2.1 points at budget 253/503, versus 6.5/11.7 for Metattack at comparable budgets. The edges that fool the model (dissimilar) are exactly the ones Jaccard removes, while the edges that evade Jaccard (similar) barely fool the model — attack potency and detectability are coupled. (Jaccard even slightly *hurts* here, paying its clean-accuracy cost while removing none of the evasion edges.) This is a heuristic, not gradient-optimal, adaptive attack; a jointly optimized adaptive attack remains future work.

8 Discussion

The grid answers RQ2 directly. The attacker mostly *adds* dissimilar edges; GCN-Jaccard *removes* exactly those (which is why removal defends); FoSR *adds* more edges; ProxyDelete *removes* edges by a spectral criterion. *FoSR vs. ProxyDelete* isolates edge direction; *ProxyDelete vs. GCN-Jaccard* isolates the removal criterion. Our working hypothesis was that add-based rewiring is neutral-to-harmful for robustness while remove-based pruning partially transfers. The data only half match it: add-based FoSR is indeed neutral-to-harmful, but *remove-based spectral pruning does not transfer either* — ProxyDelete behaves like the vanilla GCN, not like the defense. The discriminating factor is therefore not the add/remove *direction* but the removal *criterion*: only similarity-targeted removal (Jaccard), which directly undoes the attacker’s dissimilar-edge insertions — deleting $\sim 50\%$ of them versus near-chance for the spectral and random arms (Table 4) — improves robustness, while spectral-gap-guided editing (add or remove) does not. On this testbed the conceptual bridge between over-squashing and adversarial robustness is *not* supported: mitigating over-squashing does

not buy robustness. A natural worry is budget: Jaccard removes ~ 643 edges (similarity-threshold-driven) versus the 253 matched across the spectral/random arms. Equalizing it does not change the verdict — at a 643-edge budget, ProxyDelete still reaches only 76.9/71.3% robust (vs. Jaccard’s 78.8/74.5%), while removing 643 random edges (DropEdge, 76.5/70.9) or adding 643 (FoSR, 76.3/69.9) does not help and can hurt. The gap to Jaccard is therefore driven by the removal *criterion*, not the edge budget.

9 Conclusion

On a controlled add-vs-remove grid (Cora, Metattack), over-squashing mitigations did not improve adversarial robustness: add-based FoSR and spectral remove-based ProxyDelete both tracked the undefended GCN ($\approx 77\%/72\%$ robust at ptb 0.05/0.10), while only purpose-built, attack-aware defenses did — GCN-Jaccard (similarity pruning) and GCN-SVD (low-rank filtering) — through two distinct mechanisms. The discriminating factor is the editing *criterion*, not the add/remove direction or edge count: robustness here comes from targeting the attacker’s perturbation (dissimilar edges / high-rank structure), not from optimizing the spectral gap. On this testbed, then, the over-squashing→robustness bridge is not supported — an honest negative result, and one that replicates on Citeseer (Table 5). A heuristic defense-aware attacker can evade Jaccard’s filter but only by becoming much weaker, suggesting attack potency and detectability are coupled. The Jacobian probe confirms over-squashing is present (sensitivity decays $\sim 5\times$ per hop) yet shows poisoning does not steepen that decay (RQ1), reinforcing that the two phenomena are decoupled. Future work: a gradient-optimal adaptive attack, and testing whether a single Jacobian measure tracks both phenomena (RQ3).

Reproducibility. All numbers are produced by fixed-seed scripts and are reproducible end-to-end; poisoned graphs are cached and every table regenerates from a single command. We report only real, measured results.

References

- [1] Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations (ICLR)*, 2021.
- [2] Hugo Attali, Davide Buscaldi, Nathalie Pernelle, and Fragkiskos D. Malliaros. Graph rewiring in GNNs to mitigate over-squashing and over-smoothing: A survey. *arXiv preprint arXiv:2411.17429*, 2024. Accepted at IJCAI 2026, Survey Track.
- [3] Francesco Di Giovanni, Lorenzo Giusti, Federico Barbero, Giulia Luise, Pietro Lio, and Michael M. Bronstein. On over-squashing in message passing neural networks: The impact of width, depth, and topology. In *International Conference on Machine Learning (ICML)*, 2023.
- [4] Philipp Foth, Lukas Gosch, Simon Geisler, Leo Schwinn, and Stephan Günnemann. Adversarial robustness of graph transformers. *Transactions on Machine Learning Research (TMLR)*, 2025.
- [5] Sujia Huang, Lele Fu, Zhen Cui, Tong Zhang, Na Song, and Bo Huang. GraphTorque: Torque-driven rewiring graph neural network. *arXiv preprint arXiv:2507.21422*, 2025.
- [6] Adarsh Jamadandi, Celia Rubio-Madrigal, and Rebekka Burkholz. Spectral graph pruning against over-squashing and over-smoothing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

- [7] Wei Jin, Yao Ma, Xiaorui Liu, Xianfeng Tang, Suhang Wang, and Jiliang Tang. Graph structure learning for robust graph neural networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2020.
- [8] Kedar Karhadkar, Pradeep Kr. Banerjee, and Guido Montúfar. FoSR: First-order spectral rewiring for addressing oversquashing in GNNs. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [10] Niklas Kormann, Benjamin Doerr, and Johannes F. Lutzeyer. Position: Don’t be afraid of over-smoothing and over-squashing. *arXiv preprint arXiv:2601.07419*, 2026.
- [11] Yaxin Li, Wei Jin, Han Xu, and Jiliang Tang. DeepRobust: A PyTorch library for adversarial attacks and defenses. *arXiv preprint arXiv:2005.06149*, 2020.
- [12] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M. Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations (ICLR)*, 2022.
- [13] Huijun Wu, Chen Wang, Yuriy Tyshetskiy, Andrew Docherty, Kai Lu, and Liming Zhu. Adversarial examples for graph data: Deep insights into attack and defense. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
- [14] Daniel Zügner and Stephan Günnemann. Adversarial attacks on graph neural networks via meta learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- [15] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. Adversarial attacks on neural networks for graph data. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2018.

A Implementation Notes

DeepRobust (0.2.9) installed with `-no-deps` (to avoid a broken `gensim` pin) plus prebuilt PyG extension wheels (`torch_sparse`, `torch_scatter`) matched to `torch 2.9/cu128`; all compute on CPU. Full Metattack (Meta-Self) on Cora costs ~ 37 min at `ptb = 0.05` on CPU, hence the precompute-and-cache design.

B Budget-matched robustness check

To rule out that Jaccard’s advantage is merely a larger edit budget, we re-ran the remove/add arms with a budget of 643 edges (matching Jaccard’s similarity-threshold count) instead of the 253 used in Table 3. Robust accuracy is essentially unchanged: the criterion, not the budget, drives the defense.

Method (budget 643)	Clean	Robust@0.05	Robust@0.10
GCN (vanilla)	83.50	77.05	71.85
GCN-Jaccard (~ 643)	82.16	78.84	74.45
ProxyDelete (remove)	82.03	76.89	71.28
DropEdge (random)	81.66	76.48	70.88
FoSR (add)	82.26	76.32	69.89

Table 6: Budget-matched check (Cora, test acc %, 3 seeds). Even at Jaccard’s ~ 643 -edge budget, spectral/random/add arms stay at the vanilla-GCN level; only Jaccard defends.